Airlles/Hani Notes

Python Crash Course

Published: September 27, 2025

Disclaimer & Author's Note

*This crash course is a personal set of notes I compiled while learning Python through the **NeetCode crash course** and other resources. It is not original instructional content but rather a structured summary of what I learned, organized into a concise reference format. The purpose of this document is educational and for personal use, but I am sharing it publicly in case others also find it useful.*

🔹 Variables

Dynamic typing → no need to declare type:

```
x = 5
x = "hello"  # reassign to different type
```

- **Increment** → `x++` ❌. Use: `x += 1` or `x = x + 1`.
- **Null** → `None` represents absence of value.

🔹 Conditionals

Syntax: no parentheses, no braces → use **indentation**.

```
if x > 5:
    print("greater")
elif x == 5:
    print("equal")
else:
    print("smaller")
```

- **Operators**: use `and` / `or` instead of `&&` / `||`.

Loops

While loop

```
while x < 5:
    print(x)
    x += 1
```

For loop with range

```
for i in range(5):      # 0..4
    print(i)

for i in range(2, 6):   # 2..5
    print(i)

for i in range(5, 0, -1):  # countdown 5..1
    print(i)
```

🔹 Math

- **Division**: `/` (decimal), `//` (integer).
- **Negative division**: wrap with `int()` for safe truncation → `int(-3/2)`.
- **Modulus**: `10 % 3`

For negatives:

```
import math
math.fmod(-10, 3)
```

- **Rounding**:
  - `math.floor(3/2)` → round down.
  - `math.ceil(3/2)` → round up.
- **Other functions**:
  - `math.sqrt(2)`
  - `math.pow(2, 3)` → 8
- **Infinity**: `float("inf")`, `float("-inf")`.

🔹 Lists (Arrays)

Initialization:

```
arr = [1, 2, 3]
```

Add & Remove:

- Append: `arr.append(4)`
- Pop: `arr.pop()`
- Insert at index: `arr.insert(1, 7)` # O(n)

- **Reassign**: `arr[0] = 99`

Default size:

```
arr = [1] * 5   # [1,1,1,1,1]
```

- **Length**: `len(arr)`
- **Negative indexing**: `arr[-1]` = last element.

Slicing

```
arr = [1,2,3,4]
print(arr[1:3])   # [2,3]
print(arr[0:4])   # [1,2,3,4]
```

🔹 List Tricks

Unpacking

```
a, b, c = [1, 2, 3]
```

Looping styles

```
# Index only
for i in range(len(nums)):
    print(nums[i])

# Value only
for n in nums:
    print(n)

# Index + value
for i, n in enumerate(nums):
    print(i, n)
```

Zip (parallel iteration)

```
nums1 = [1,2,3]
nums2 = [4,5,6]
for n1, n2 in zip(nums1, nums2):
    print(n1, n2)
```

Reverse & Sort

```
arr.reverse()
arr.sort()
arr.sort(reverse=True)
arr.sort(key=lambda x: len(x))  # by length
```

List Comprehension

```
arr = [i for i in range(5)]      # [0,1,2,3,4]
arr = [i+1 for i in range(5)]    # [1,2,3,4,5]
```

🔹 2D Lists (Matrices)

Initialization

```
matrix = [[0]*3 for _ in range(3)]
# 3x3 matrix filled with 0s
```

- ⚠️ Don't use `[[0]*3]*3` → it creates references to the same row.

1) **Access elements**

```
print(matrix[0][1])   # row 0, col 1
matrix[2][2] = 9      # update element
```

2) **Iterating**

```
for row in matrix:
    for val in row:
       print(val)
```

3) **Populate with values**

```
# Fill with row*col product
matrix = [[i*j for j in range(3)] for i in range(3)]
print(matrix)  # [[0,0,0], [0,1,2], [0,2,4]]

# Identity matrix (1s on diagonal, 0 elsewhere)
identity = [[1 if i==j else 0 for j in range(3)] for i in range(3)]
print(identity)  # [[1,0,0],[0,1,0],[0,0,1]]
```

🔹 Strings

Declaration

```
s1 = "hello"
s2 = 'world'
```

Array-like access

```
 s = "abc"
print(s[0])      # 'a'
print(s[0:2])    # 'ab'  (slice first two chars)
```

Immutable → cannot directly modify characters.

```
 s = "abc"
# s[0] = "z" ❌ (error)
```

Updating (concatenation)

```
 s = "abc"
s += "def"
print(s)  # "abcdef"
```

Type conversion

```
 num = int("123")   # 123
```

ASCII values

```
 print(ord("A"))  # 65
```

Joining strings

```
strings = ["ab", "cd"]
print("".join(strings))   # abcd
print("-".join(strings))  # ab-cd
```

Joining specific indexes

```
strings = ["a","b","c","d","e","f","g","h","i"]
print("".join([strings[0], strings[7]]))   # "ai"
print("-".join([strings[0], strings[7]]))  # "a-h"
```

🔹 Queues (double-ended queue)

Import + Initialization

```
from collections import deque
queue = deque()
```

Add to right (enqueue)

```
queue.append(1)
queue.append(2)
print(queue)   # deque([1, 2])
```

Remove from left (dequeue)

```
queue.popleft()
print(queue)   # deque([2])
```

Add to left

```
queue.appendleft(1)
print(queue)   # deque([1, 2])
```

Remove from right

```
queue.pop()
print(queue)   # deque([1])
```

🔹 Hash Sets

Initialization + Add

```
mySet = set()
mySet.add(1)
mySet.add(2)
print(mySet)   # {1, 2}
```

Length + Membership

```
print(len(mySet))   # 2
print(1 in mySet)   # True
print(3 in mySet)   # False
```

Remove values

```
mySet.remove(2)
print(mySet)   # {1}
```

Convert list → set

```
nums = [1,2,3,3]
print(set(nums))   # {1,2,3} (removes duplicates)
```

Set comprehension

```
mySet = {i for i in range(5)}
print(mySet)   # {0,1,2,3,4}
```

1) **Fast membership checks** → O(1) average time for `x in mySet`.
   E.g., quickly test if you've seen a number before.

2) **Remove duplicates** from a list while preserving uniqueness.

3) **Detecting cycles or repeats** in problems like linked list cycle detection or repeated substring patterns.

4) **Storing visited states** in graph/DFS/BFS problems to avoid infinite loops.

5) **Intersection / Union of groups** (e.g., common elements between two lists).

🔹 Hash Map (aka dict)

Initialization + Insert

```
myMap = {}
myMap["alice"] = 88
myMap["bob"] = 77
print(myMap)          # {'alice': 88, 'bob': 77}
print(len(myMap))     # 2
```

Update values

```
myMap["alice"] = 80
print(myMap["alice"])   # 80
```

Membership + Remove

```
print("alice" in myMap)   # True
myMap.pop("alice")
print("alice" in myMap)   # False
```

Literal initialization

```
myMap = {"alice": 90, "bob": 70}
print(myMap)
```

Dict comprehension

```
myMap = {i: 2*i for i in range(3)}
print(myMap)   # {0:0, 1:2, 2:4}
```

Looping

```
myMap = {"alice": 90, "bob": 70}
for key in myMap:
    print(key, myMap[key])
    # alice 90
    # bob 70

for val in myMap.values():
    print(val)
    # 90
    # 70

for key, val in myMap.items():
    print(key, val)
    # alice 90   # bob 70
```

🔹 Tuples (will be used as keys for a hash map/set)

Definition → ordered, immutable sequence (like a list !change)

```
tup = (1, 2, 3)
print(tup)      # (1, 2, 3)
print(tup[0])   # 1
print(tup[-1])  # 3
```

Immutability → you cannot modify elements.

```
tup[0] = 99   # ❌ TypeError
```

As keys in dict / set

```
myMap = {(1, 2): 3}
print(myMap[(1, 2)])    # 3
mySet = set()
mySet.add((1, 2))
print((1, 2) in mySet)  # True
```

✅ Works because tuples are **hashable** (immutable).

Lists cannot be keys

```
myMap = {}
myMap[[3,4]] = 5   # ❌ TypeError: unhashable type: 'list'
```

Use Cases

- **Grid problems** → store coordinates `(row, col)` in a set to track visited cells.
- **Hash map keys with multiple attributes** → e.g., `myMap[(user_id, timestamp)] = "data"`.

Return multiple values from a function (Python returns tuples by default):

```
 def getMinMax(nums):
    return min(nums), max(nums)
low, high = getMinMax([1,5,9])
print(low, high)   # 1 9
```

🔹 Heaps

Min Heap

```
import heapq
minHeap = []
heapq.heappush(minHeap, 3)
heapq.heappush(minHeap, 2)
heapq.heappush(minHeap, 4)
print(minHeap[0])   # 2  (minimum value)

# Print smallest → largest
while len(minHeap):
    print(heapq.heappop(minHeap))
    # 2, 3, 4
```

Max Heap (simulated with negatives)

```
import heapq
maxHeap = []
heapq.heappush(maxHeap, -3)
heapq.heappush(maxHeap, -2)
heapq.heappush(maxHeap, -4)
print(-1 * maxHeap[0])   # 4  (maximum value)

# Print largest → smallest
while len(maxHeap):
    print(-1 * heapq.heappop(maxHeap))
    # 4, 3, 2
```

Heapify Existing Array

```
import heapq

arr = [2, 1, 8, 4, 5]
heapq.heapify(arr)   # turn arr into min-heap in O(n)

while arr:
    print(heapq.heappop(arr))
    # 1, 2, 4, 5, 8
```

🔹 Functions

Basic function definition

```
def myFunc(n, m):
    return n * m
print(myFunc(3, 4))   # 12
```

Nested Functions

Inner functions **have access to outer variables** (closure).

```
def outer(a, b):
    c = "c"
    def inner():
        return a + b + c
    return inner()

print(outer("1", "2"))   # "12c"
```

Modifying Objects vs Variables

- **Lists (mutable objects)** can be modified inside a nested functions
- **Primitive values** (like int/str) cannot be reassigned unless you use `nonlocal`.

```
def double(arr, val):
    def helper():
        # Modify list in-place (works)
        for i, n in enumerate(arr):
            arr[i] *= 2

        # val here is treated as local → doesn't affect outer val
        # val *= 2   ❌

        # Use nonlocal to modify outer val
        nonlocal val
        val *= 2

    helper()
    print(arr, val)

nums = [1, 2]
val = 3
double(nums, val)   # [2, 4], 6
```

🔹 Classes

Basic class with constructor

```
class MyClass:
    # Constructor
    def __init__(self, nums):
        self.nums = nums              # store list
        self.size = len(nums)         # store length

    # Method: get length
    def getLength(self):
        return self.size

    # Method: call another method
    def getDoubleLength(self):
        return 2 * self.getLength()
```

Usage

```
obj = MyClass([1, 2, 3, 4])
print(obj.getLength())        # 4
print(obj.getDoubleLength())  # 8
```

Key Points

- **`__init__`** → constructor, called when you make an object.
- **`self`** → always required as the first parameter; refers to the object itself.
- **Attributes** → defined with `self.<name>` (like `self.nums`, `self.size`).
- **Methods** → functions inside the class that act on the object.
- Methods can call **other methods** via `self.getLength()`.

Thank you for taking the time to read through this crash course.

Everything in this guide is based on the excellent NeetCode Python Crash Course video. I simply took the material and rewrote it into my own structured notes so I could reference it quickly while learning.

I am not the original author of this content, full credit goes to NeetCode. This document exists only as my personal study notes, which I'm sharing publicly in case others find it helpful as a compact reference.

If you want the full explanations and teaching, please check out the original NeetCode channel.

Happy coding, and best of luck in your learning journey :)